

Bio-Analytics Project

DNA Sequence Quality Control — Complete Roadmap

Project Overview

You are building a Python script that analyzes a DNA sequence string (made of A, T, G, C characters) and calculates its GC-content — the percentage of Guanine (G) and Cytosine (C) bases. High GC-content typically indicates greater DNA stability. The project is broken into 4 steps, each building on the last.

Project File Structure

```
dna-qc/  
■■■ dna_analyzer.py      # Core logic (your main script)  
■■■ test_dna.py          # Tests using pytest  
■■■ requirements.txt     # biopython, pytest  
■■■ README.md           # GitHub project description
```

The 4 Steps

Step 1	GC-Content Calculator	To Do
Tasks:	■ Write <code>gc_content(sequence)</code> function ■ Count G and C characters in the string ■ Divide by total length $\times 100$ to get a percentage ■ Return the result ■ Test it manually by calling the function with sample DNA	
Commit:	<code>git commit -m "feat: add gc_content function"</code>	

Key concepts: strings, loops or `.count()`, basic math, functions

Step 2	Input Validation	To Do
--------	------------------	-------

Tasks:	■ Write <code>validate_sequence(sequence)</code> function ■ Reject sequences with invalid characters (not A/T/G/C) ■ Handle empty strings gracefully ■ Handle lowercase input (hint: use <code>.upper()</code>) ■ Return True/False or raise an exception
Commit:	<code>git commit -m "feat: add input validation"</code>

Key concepts: sets, conditionals, raising exceptions, edge cases

Step 3	Quality Verdict	To Do
Tasks:	■ Write <code>quality_verdict(gc_percent)</code> function ■ Return 'Stable' if GC $\geq$ 50%, 'Unstable' if below ■ Connect all 3 functions into a <code>main()</code> function ■ Print a clean summary: sequence, GC%, and verdict ■ Write tests in <code>test_dna.py</code> using <code>pytest</code>	
Commit:	<code>git commit -m "feat: add quality verdict and main runner"</code>	

Key concepts: conditionals, function composition, `pytest` basics

Step 4	Biopython Refactor (Bonus)	To Do
Tasks:	■ Install <code>biopython</code>: <code>pip install biopython</code> ■ Add <code>biopython</code> to <code>requirements.txt</code> ■ Import <code>Seq</code> from <code>Bio.Seq</code> and <code>GC</code> from <code>Bio.SeqUtils</code> ■ Rewrite <code>gc_content()</code> using <code>Biopython</code>'s built-in <code>GC()</code> ■ Compare your manual result vs <code>Biopython</code>'s result	
Commit:	<code>git commit -m "feat: refactor with biopython"</code>	

Key concepts: external libraries, refactoring, `Biopython` basics — great resume addition!

Libraries & Installation

Library	Purpose	Install Command
<code>biopython</code>	Professional DNA tools (Step 4)	<code>pip install biopython</code>
<code>pytest</code>	Run your tests (Step 3)	<code>pip install pytest</code>

GitHub Workflow Reminder

After completing each step, run these commands:

```
git status          # See what changed
git add .           # Stage all changes
git commit -m "feat: ..." # Save a snapshot
git push            # Upload to GitHub
```

Tip: Commit after each step — not just at the end. A visible commit history makes your GitHub profile look great to employers.